Sardar Patel Institute of Technology

Bhavan's Campus, Munshi Nagar, Andheri (West), Mumbai-400058-India
(Autonomous Institute Affiliated to University of Mumbai)

Department of Computer Engineering
Generative AI Lab Manual AY 2026-27

Class: BTech(CE) | **Semester: VII**
Course Name: Generative AI (PE III) | **Course Code: CE413**
Faculty Name: Prof. Yogita Patil | **Academic Year: 2026-27**

Experiment 2: Design, Implementation, and Comparative Analysis of Generative Image Models

1. Title

Design, Implementation, and Comparative Evaluation of a Generative Image Model

2. Aim

To design, implement, train, and evaluate a Generative Image Model — chosen from an Autoencoder, Variational Autoencoder (VAE), Generative Adversarial Network (GAN), Diffusion Model, or Transformer-based Image Generator — on a selected image dataset, and to investigate the effect of at least one architectural or hyperparameter modification on image quality, convergence behavior, and computational performance through comparative analysis.

3. Problem Statement

Design, implement, and evaluate a generative image model (Autoencoder, Variational Autoencoder, GAN, Diffusion Model, or Transformer-based Image Generator) for a selected image dataset. Investigate the effect of at least one architectural or hyperparameter modification on image quality, convergence, and computational performance through comparative analysis.

4. Course Outcomes (COs) Mapped

After successful completion of this experiment, students will be able to:

- CO1: Understand the working principles of generative image models.
- CO2: Implement deep generative models using modern deep learning frameworks.
- CO3: Analyze model performance using qualitative and quantitative evaluation metrics.
- CO4: Compare different model configurations and justify the impact of architectural modifications.

5. Prerequisites

Students should be familiar with:

- Python programming
- Deep learning fundamentals
- Neural networks
- Convolutional Neural Networks (CNN)
- PyTorch or TensorFlow
- Image preprocessing techniques
- Basic probability concepts (required for VAE — mean, variance, sampling)
- GPU computing (optional but recommended for faster training)

6. Software and Hardware Requirements

6.1 Software

- Python 3.10+
- Jupyter Notebook / Google Colab
- TensorFlow 2.x or PyTorch
- NumPy
- Matplotlib
- OpenCV
- torchvision / tensorflow-datasets
- scikit-learn

6.2 Hardware

- Minimum 8 GB RAM
- GPU (recommended; CUDA-enabled for PyTorch/TensorFlow acceleration)
- Internet connection (for downloading datasets and pretrained weights)

7. Learning Objectives

After performing this experiment, students will be able to:

1. Understand different generative image models and their underlying principles.
2. Implement one image generation model end to end.
3. Train the model on a chosen image dataset.
4. Generate new, realistic images from the trained model.
5. Compare different model configurations systematically.
6. Evaluate image quality using appropriate quantitative and qualitative metrics.
7. Analyze computational efficiency in terms of time, memory, and parameters.

8. Theoretical Background

8.1 Introduction

Generative Artificial Intelligence focuses on learning the underlying probability distribution of the training data so that new, previously unseen samples can be generated. This is fundamentally different from discriminative modeling.

Discriminative models learn a decision boundary p(y|x) to predict a label y given input x (e.g., classifying an image as a cat or dog). Generative models instead learn p(x), or a joint distribution p(x, y), so that new samples resembling the training data can be drawn from the learned distribution.

Applications of generative image models include:

- Face generation
- AI art and creative content generation
- Medical image synthesis (e.g., augmenting scarce clinical imaging data)
- Image super-resolution
- Data augmentation for downstream classifiers
- Image restoration and inpainting

8.2 Common Generative Models

8.2.1 Autoencoder (AE)

An autoencoder is composed of three parts:

- Encoder — compresses the input image into a lower-dimensional latent representation.
- Latent Space — the compressed bottleneck representation of the image.
- Decoder — reconstructs the image from the latent vector.

Loss Function:

```
Loss = Mean Squared Error (MSE) between input image and reconstructed image
```

Advantages:

- Simple to implement
- Fast training

Limitations:

- Produces blurry images
- Limited generative capability — a plain AE's latent space is not structured for sampling new, realistic images

8.2.2 Variational Autoencoder (VAE)

Instead of mapping an input to a single fixed latent vector, a VAE learns a probability distribution over the latent space.

The encoder predicts two vectors:

- Mean (μ)
- Variance / log-variance (σ)

A latent vector is then sampled using the reparameterization trick:

$$z = \mu + \sigma \odot \varepsilon, \quad \text{where } \varepsilon \sim N(0, I)$$

The loss function consists of two terms:

- Reconstruction Loss — measures how well the decoded image matches the input
- KL Divergence — regularizes the learned latent distribution to stay close to a standard normal distribution N(0, I), enabling smooth sampling

Advantages:

- Smooth, continuous latent space
- Better interpolation between samples

8.2.3 Generative Adversarial Network (GAN)

A GAN consists of two competing neural networks trained simultaneously:

- Generator — takes random noise as input and generates a fake image.
- Discriminator — classifies an input image as Real (from the dataset) or Fake (produced by the generator).

The two networks compete in a minimax game: the generator tries to fool the discriminator, while the discriminator tries to correctly distinguish real from fake images. Over training, the generator learns to produce increasingly realistic images.

Advantages:

- High-quality, sharp image outputs

Disadvantages:

- Difficult and unstable training
- Mode collapse — the generator produces limited variety of outputs

8.2.4 Diffusion Models

Training (Forward Process):

Images are gradually corrupted by adding random noise over a sequence of timesteps until they become pure noise.

Generation (Reverse Process):

A neural network (typically a U-Net) is trained to reverse this process — learning to remove noise step by step, starting from pure noise, until a realistic image emerges.

Advantages:

- Excellent, state-of-the-art image quality
- Stable training compared to GANs

Disadvantages:

- Computationally expensive — requires many sequential denoising steps for both training and inference

8.2.5 Transformer-based Image Generators

Examples: ImageGPT, DALL·E, MaskGIT.

These models use self-attention mechanisms to model long-range dependencies between pixels or image patches/tokens, rather than relying purely on local convolutional receptive fields.

Suitable for:

- Large-scale datasets
- High-resolution image generation

9. Dataset Selection

Students may select any one of the following datasets for the experiment:

Dataset	Images	Classes	Image Size
MNIST	70,000	10	28×28
Fashion-MNIST	70,000	10	28×28
CIFAR-10	60,000	10	32×32
CelebA	202,599	Faces	64×64 and above
Oxford Flowers	8,189	102	Variable

Selection guidance:

- MNIST / Fashion-MNIST — best for quick prototyping and grayscale, low-resolution models (AE, VAE, small GAN).
- CIFAR-10 — moderate complexity; good for evaluating convolutional GANs and VAEs on RGB images.
- CelebA — suited for face-generation GANs/VAEs; larger dataset, higher computational cost.
- Oxford Flowers — smaller dataset useful for fine-grained, variable-size image generation experiments.

10. Reference Architecture (GAN Example)

A typical DCGAN-style architecture used as a reference implementation:

```
Generator:
  Input Noise Vector (dim = 100)
        |
  Dense Layer  →  Reshape to (H0 × W0 × C0)
        |
  ConvTranspose2D → BatchNorm → ReLU   (repeated, upsampling)
        |
  Output Layer (Tanh activation) → Generated Image

Discriminator:
  Generated / Real Image
        |
  Conv2D → LeakyReLU → BatchNorm   (repeated, downsampling)
        |
  Flatten → Dense → Sigmoid  →  Real / Fake probability
```

11. Experimental Procedure

Step 1 — Select Dataset

Choose one image dataset from Section 9 based on available compute resources and the generative model to be implemented.

Step 2 — Preprocess the Images

Resize images to a fixed dimension; normalize pixel values (typically to [-1, 1] or [0, 1]); convert images into tensors compatible with the chosen framework (PyTorch tensors / TensorFlow tensors).

Step 3 — Design a Generative Model

Select and implement one architecture: Autoencoder, VAE, GAN, or Diffusion Model, based on the encoder-decoder / generator-discriminator / noise-denoising principles covered in Section 8.

Step 4 — Initialize Model Parameters

Set baseline hyperparameters, e.g., Learning Rate = 0.0002, Batch Size = 64, Epochs = 50, Optimizer = Adam ($\beta1$ = 0.5, $\beta2$ = 0.999 for GANs).

Step 5 — Train the Baseline Model

Train the model on the preprocessed dataset. Record training loss, validation loss (where applicable), and time taken per epoch.

Step 6 — Generate Sample Images

Periodically sample images from the model (e.g., every 5–10 epochs) and save them to visually track training progress.

Step 7 — Modify One Parameter

Introduce exactly one architectural or hyperparameter modification (see options below) to create a second, modified model configuration.

Step 8 — Retrain the Modified Model

Train the modified model under identical dataset, preprocessing, and (where not the variable under test) hyperparameter conditions.

Step 9 — Compare Both Models

Evaluate the baseline and modified models side by side using the metrics in Section 13 and the comparison table in Section 14.

Step 10 — Analyze Results

Interpret the observed differences in image quality, convergence speed, and computational cost, and relate them back to the theoretical expectations from Section 8.

11.1 Modification Options

Choose exactly one of the following categories for Step 7:

Architecture Modification

- Increase the latent dimension
- Add additional convolutional layers
- Introduce residual blocks
- Add a self-attention layer
- Add / remove batch normalization
- Introduce dropout

Hyperparameter Modification

- Learning rate
- Batch size
- Optimizer (e.g., Adam vs. RMSProp vs. SGD)
- Number of training epochs
- Latent vector size

12. Sample Code Snippets

The following minimal PyTorch snippets illustrate the core building blocks. Students should adapt these to their chosen dataset, framework, and model.

12.1 Data Loading and Preprocessing

```
import torch
from torchvision import datasets, transforms

transform = transforms.Compose([
    transforms.Resize((64, 64)),
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))  # scales to [-1, 1]
])

dataset = datasets.CIFAR10(root='./data', train=True, download=True, transform=transform)
dataloader = torch.utils.data.DataLoader(dataset, batch_size=64, shuffle=True)
```

12.2 Simple Autoencoder

```
import torch.nn as nn

class Autoencoder(nn.Module):
    def __init__(self, latent_dim=64):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Conv2d(3, 32, 4, 2, 1), nn.ReLU(),
            nn.Conv2d(32, 64, 4, 2, 1), nn.ReLU(),
            nn.Flatten(),
            nn.Linear(64 * 16 * 16, latent_dim)
        )
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 64 * 16 * 16), nn.ReLU(),
            nn.Unflatten(1, (64, 16, 16)),
            nn.ConvTranspose2d(64, 32, 4, 2, 1), nn.ReLU(),
            nn.ConvTranspose2d(32, 3, 4, 2, 1), nn.Tanh()
        )

    def forward(self, x):
        z = self.encoder(x)
        return self.decoder(z)
```

12.3 GAN Training Loop (Skeleton)

```
criterion = nn.BCELoss()
optim_G = torch.optim.Adam(generator.parameters(), lr=0.0002, betas=(0.5, 0.999))
optim_D = torch.optim.Adam(discriminator.parameters(), lr=0.0002, betas=(0.5, 0.999))

for epoch in range(num_epochs):
    for real_images, _ in dataloader:
```

```
        # Train Discriminator
        noise = torch.randn(batch_size, 100, 1, 1)
        fake_images = generator(noise)
        d_loss_real = criterion(discriminator(real_images), real_labels)
        d_loss_fake = criterion(discriminator(fake_images.detach()), fake_labels)
        d_loss = d_loss_real + d_loss_fake
        optim_D.zero_grad(); d_loss.backward(); optim_D.step()

        # Train Generator
        g_loss = criterion(discriminator(fake_images), real_labels)  # wants D to say 'real'
        optim_G.zero_grad(); g_loss.backward(); optim_G.step()
```

12.4 VAE Loss Function

```
def vae_loss(recon_x, x, mu, logvar):
    recon_loss = nn.functional.mse_loss(recon_x, x, reduction='sum')
    kl_div = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
    return recon_loss + kl_div
```

13. Evaluation Metrics

13.1 Image Quality Metrics

Metric	Applicable To	Interpretation
Mean Squared Error (MSE)	AE / VAE	Lower is better — pixel-level reconstruction error
Structural Similarity Index (SSIM)	AE / VAE / GAN	Higher is better (max 1) — perceptual structural similarity
Peak Signal-to-Noise Ratio (PSNR)	AE / VAE	Higher is better — reconstruction fidelity in dB
Fréchet Inception Distance (FID)	GAN / Diffusion	Lower is better — distributional distance between real and generated features
Inception Score (IS)	GAN / Diffusion	Higher is better — measures quality and diversity

Visual Inspection

In addition to quantitative metrics, students should visually inspect generated samples and comment on:

- Sharpness

- Diversity across generated samples
- Presence of artifacts or distortions
- Color consistency

13.2 Performance Metrics

In addition to image quality, the following computational metrics should be measured and recorded:

- Training time (total and per epoch)
- Inference time (per generated sample / per batch)
- GPU memory usage
- Number of trainable parameters
- Epoch-wise loss curves
- Convergence speed (epochs to reach a stable loss plateau)

14. Comparative Analysis Framework

Students must complete the following comparison table for the Baseline Model (Model 1) versus the Modified Model (Model 2):

Parameter	Model 1 (Baseline)	Model 2 (Modified)
Architecture		
Learning Rate		
Batch Size		
Epochs		
Training Time		
Final Loss		
FID Score		
SSIM		
Generated Image Quality		

Parameter	Model 1 (Baseline)	Model 2 (Modified)
GPU Memory		

15. Expected Observations

Based on the theory and typical experimental results, students should expect to observe:

- Faster convergence does not always translate into better image quality.
- Increasing latent dimensions generally improves diversity of generated images.
- Larger batch sizes improve training stability but require more GPU memory.
- GANs typically produce sharper images than plain Autoencoders.
- Diffusion models generate the highest-quality images but at the highest computational cost.
- VAEs produce smoother, more structured latent spaces but slightly blurrier outputs compared to GANs.

16. Sample Results Table

Record training progress at regular intervals:

Epoch	Training Loss	Validation Loss	Time/Epoch (sec)
1			
10			
20			
30			
40			
50			

17. Generated Image Comparison — Reporting Format

In the lab report, students should attach and clearly label the following image sets:

- Original (real) dataset images
- Generated images from the Baseline Model

- Generated images from the Modified Model

For each set, include a short written observation covering:

- Sharpness
- Diversity
- Visual realism
- Reconstruction quality (where applicable, e.g., AE/VAE)

18. Precautions and Good Practices

- Always normalize input images consistently between training and inference.
- Fix a random seed for reproducibility when comparing baseline and modified models.
- Save model checkpoints periodically to avoid losing progress on long training runs.
- Monitor GPU memory usage to avoid out-of-memory (OOM) crashes, especially with larger batch sizes.
- Change only one variable at a time between the baseline and modified experiment for a valid comparison.
- Use the same dataset split and preprocessing pipeline for both models being compared.

19. Common Errors and Troubleshooting

Issue	Likely Cause	Suggested Fix
Blurry generated images	Using AE/VAE with only MSE loss	Try a GAN/diffusion model, or add perceptual/adversarial loss
Mode collapse (GAN)	Generator finds a narrow set of outputs that fool the discriminator	Reduce learning rate, add label smoothing, use minibatch discrimination or feature matching
Loss oscillating / not converging (GAN)	Imbalanced generator/discriminator training	Adjust relative learning rates, add batch normalization, use Wasserstein loss with gradient penalty
CUDA out-of-memory	Batch size or model too large for available GPU memory	Reduce batch size, use mixed-precision training, or use gradient accumulation
NaN loss values	Learning rate too high, unstable gradients	Reduce learning rate, add gradient clipping, check for numerical instability in loss functions

Sardar Patel Institute of Technology

Bhavan's Campus, Munshi Nagar, Andheri (West), Mumbai-400058-India
(Autonomous Institute Affiliated to University of Mumbai)

Questions

Basic	**Intermediate**	**Advanced**
1. Why do Autoencoders compress data? 2. What is the difference between an Autoencoder and a Variational Autoencoder? 3. What is latent space? 4. Why is image normalization necessary before training?	5. Why does a GAN require both Generator and Discriminator? 6. What is mode collapse in GANs? 7. Differentiate between Binary Cross-Entropy Loss and Mean Squared Error Loss. 8. Why is LeakyReLU preferred over ReLU in GANs?	9. Why are Diffusion Models generally more stable than GANs? 10. Explain the Fréchet Inception Distance (FID). 11. How does increasing the latent dimension affect generated images? 12. Why do GANs sometimes generate blurry or distorted images?